Sumário

1. Introdução;
2. Conceitos Básicos;
3. Vantagens e Desvantagens da Programação Paralela;
4. Fatores que Afetam a Programação Paralela;
5. Quatro Passos para a Paralelização
   5.1. Análise e Dependências;
   5.2. Projeto e Implementação
   5.3. Testes de Correção;
   5.4. Análise de Desempenho;
6. Metodologia PCAM;
7. Conclusão;
8. Referências;

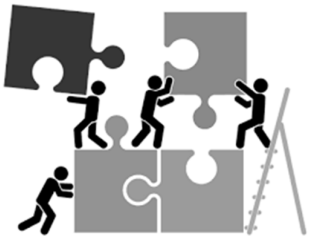
Quatro Passos para a Paralelização



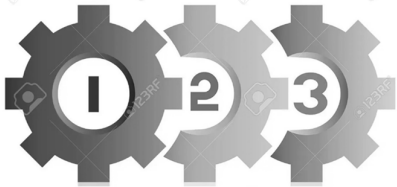
1. Análise de Dependências



3. Testes de Correção

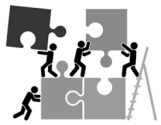


2. Projeto e Implementação Paralela



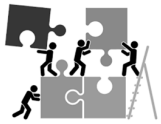
4. Análise de Desempenho

1. Análise de Dependências



- ✓ A implementação de paralelismo implica na execução simultânea de trechos de códigos;
- ✓ A técnica é especialmente útil quando aplicada em um programa que sofrerá uma **decomposição de domínio**;
  - Paralelismo através da divisão de dados a serem processados por diferentes unidades de processamento;
- ✓ É muito utilizada, também, por compiladores para fazer a paralelização automática;

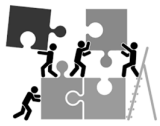
1. Análise de Dependências



- ✓ Analisar e avaliar a versão sequencial;
  - A. Identificar os possíveis pontos onde se possa implementar a execução simultânea;

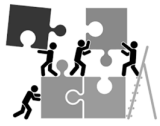
    A.1. Só poderá ser implementada em pontos de código onde NÃO haja dependência de dados;
  - B. Definir o tipo de decomposição de domínio;
  - C. Encontrar *hot spots* (trechos de código onde haja muito processamento).
    - Normalmente laços.

1. Análise de Dependências



- ✓ Analisar e avaliar a versão sequencial;
  - A. Identificar os pontos onde se possa implementar a execução simultânea;
    - É preciso observar, **com atenção,** variáveis compartilhadas que sofrem **operações de leitura e escrita** dentro dos trechos a serem paralelizados;
    - Busca-se essas variáveis no código e tenta-se realizar a refatoração (transformações no código) a fim de eliminá-las ou protegê-las do acesso concorrente;

1. Análise de Dependências



- ✓ Analisar e avaliar a versão sequencial;
  - A. Identificar os pontos onde se possa implementar a execução simultânea;

Código 01

```
for(i=0; i<N; i++) {
    X[i] = Y[i] + Z[i];
    A[i] = Y[i] + delta;
}
```

Não possui nenhuma dependência.

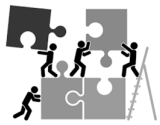
Código 02

```
SUM = 0;
for(i=0; i<N; i++) {
    X[i] = Y[i] + Z[i];
    A[i] = X[i] + delta;
    SUM += A[i];
}
```

Sofre operações de leitura e escrita (+=).

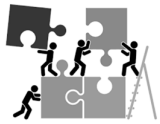
Aparece à esquerda e à direita da atribuição (escrita e leitura).

1. Análise de Dependências

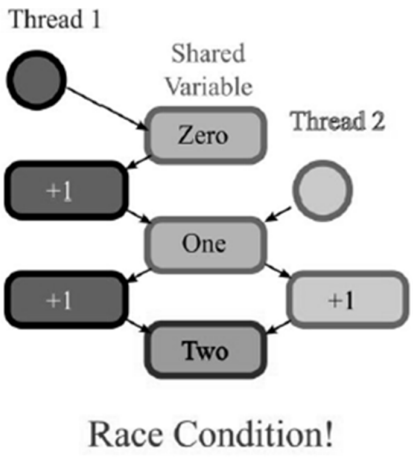


- ✓ Analisar e avaliar a versão sequencial;
  - A. Identificar os pontos onde se possa implementar a execução simultânea;
    - A utilização de mecanismos de exclusão mútua serve para evitar problemas conhecidos como condição de corrida (*race condition*);
    - As chamadas regiões ou seções críticas controlam a execução, de forma que apenas um trecho de código concorrente realize o acesso de cada vez;
    - Esse mecanismo traz benefícios em termos de execução de código (sequencial), porém pode limitar o paralelismo (deve ser evitados);

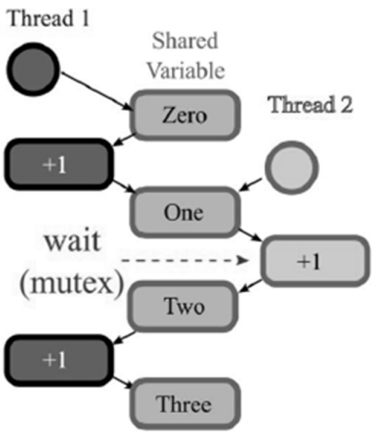
1. Análise de Dependências



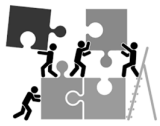
- ✓ Analisar e avaliar a versão sequencial;
  - A. Identificar os pontos onde se possa implementar a execução simultânea;



Race Condition!

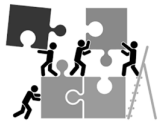


1. Análise de Dependências



- ✓ Analisar e avaliar a versão sequencial;
  - A. Identificar os pontos onde se possa implementar a execução simultânea;
    - O fato de haver dependências não significa, necessariamente, que o fragmento de código não possa ser transformado e executado de forma simultânea;
      - Deve-se verificar que tipo de dependência se trata;
      - Algumas formas não permitem a paralelização do código;
    - De acordo com *Dow* e *Severance* (1998), as dependências podem ser classificadas em 3 tipos:

1. Análise de Dependências



i. **Dependência de Fluxo** (**RAW –** *Read After Write*): a leitura depois da escrita pode ocorrer, quando uma variável sofre operação de escrita em uma primeira instrução (atribuição) e, subsequentemente, é utilizada na próxima instrução (operação de leitura). O problema ocorre caso o inverso aconteça (leitura antes da escrita) por conta da concorrência.

```
SUM = 0;
for(i=0; i<N; i++) {
    X[i] = Y[i] + Z[i];
    A[i] = X[i] + delta;
    SUM += A[i];
}
```

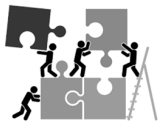
escrita

leitura

- O problema na execução ocorrerá se uma *thread* fizer a leitura antes de a referida posição do vetor sofrer a atualização necessária (escrita) na linha anterior;

- Se cada *thread* concorrente executar uma iteração completa do laço, evita-se a concorrência dentro de cada iteração;

1. Análise de Dependências



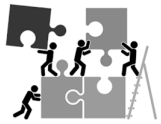
ii. **Anti-dependência** (**WAR –** *Write After Read*): a escrita depois da leitura pode ocorrer, quando uma variável sofre operação de leitura em uma primeira instrução e, subsequentemente, é atribuída na próxima instrução, o que significa sofrer operação de escrita. Novamente, o problema ocorre caso o inverso aconteça por conta da concorrência.

```
SUM = 0;
for(i=0; i<N; i++) {
    X[i] = Y[i] + Z[i];
    A[i] = X[i] + delta;
    SUM += A[i];
}
```

- **WAW:** Tem-se a ocorrência de *threads* concorrentes sobrescrevendo o valor SUM de forma sucessiva e possivelmente fora de ordem;
- **WAR:** Pode ocorrer o uso desatualizado da variável SUM em uma dada iteração *i*, caso o valor da mesma variável na iteração *i-1*, em execução por outra *thread*, não tenha sido salvo na posição de memória correspondente (tenha sido executado);

ii. **Dependência de Saída** (**WAW –** *Write After Write*): a escrita depois da escrita pode ocorrer, quando uma variável sofre operação de escrita em uma primeira instrução e, imediatamente, é atribuída com uma nova operação de escrita. Neste caso, o problema ocorre em atualizações fora da ordem sequencial imposta pela sequência de execução do laço.

1. Análise de Dependências

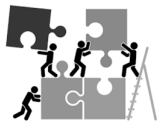


- ✓ Analisar e avaliar a versão sequencial;

  B. Detecção de *hot spots*.

  - Inicia-se a paralelização do código a partir dos trechos mais demorados (representam o "gargalo"): ***hot spots***;
  - Usualmente, os laços são bons candidatos, visto que executam tarefas repetitivas sobre um conjunto de dados homogêneos (paralelismo de dados);
    - Escolhe-se o laço candidato e, efetua-se a medição de tempo, com o objetivo de detectar aquele que consome maior tempo de processamento;
    - A medição é realizada, capturando-se o valor do relógio, antes e depois do trecho a ser medido (instrumentalização);

1. Análise de Dependências



B. Detecção de *hot spots*.

```c
#include <stdio.h>
#include <time.h>       // for clock_t, clock(), CLOCKS_PER_SEC
#include <unistd.h>     // for sleep()

#define N 10000000

// função principal para encontrar o tempo de execução de um programa em C
int main(void) {
    int k;
    double p = 1, x;

// para armazenar o tempo de execução do código
    double time_spent = 0.0;

    clock_t begin = clock();

//faz algumas coisas aqui
    x = 1.0 + (1.0/N);
        for (k=0; k<N; k++)
            p = p*x;

    clock_t end = clock();

// calcula o tempo decorrido encontrando a diferença (end - begin) e
// dividindo a diferença por CLOCKS_PER_SEC para converter em segundos

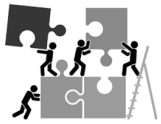
   time_spent += (double)(end - begin) / CLOCKS_PER_SEC;

   printf ("PI aproximado: %g\n", p);
   printf ("Tempo Decorrido: %f segundos\n", time_spent);

   return 0;
}
```

hot spots

1. Análise de Dependências



✓ Analisar e avaliar a versão sequencial;

B. Detecção de *hot spots*.

- A detecção de *hot spots,* com a análise de dependências, serve para identificar a viabilidade e os possíveis benefícios na paralelização de determinados trechos;
- A paralelização de código deve ser efetuado em casos críticos de tempo de processamento;
  - E onde o esforço de paralelização seja recompensado através de ganhos de desempenho;
- A análise de dependência deve indicar possíveis empecilhos que possam tornar a paralelização mais difícil;
  - Exigindo uma refatoração mais delicada;